Lab Experiment 12: Part1 - Authentication & Security (JWT)

Table of Contents

Lab Experiment 12: Part1 - Authentication & Security (JWT) 1

Lab Overview 1

Objective 1

Lab Overview

In this lab, students will continue expanding the Task Manager backend system by implementing secure user authentication using JSON Web Tokens (JWT). Students will learn how authentication systems work in modern backend applications, how passwords are securely stored using hashing, and how protected API routes are implemented.

This lab introduces critical backend security concepts used in production systems including authentication, authorization, password hashing, token-based authentication, middleware protection, and secure API design.

Students will also continue using AI-assisted development workflows while critically evaluating security-related AI-generated code.

By the end of this lab, students will have implemented a secure login system with protected API endpoints using JWT authentication.

Objective

By completing this lab, you will learn how to:

- Understand authentication vs authorization
- Securely hash passwords using bcrypt
- Generate JWT tokens during login
- Protect API routes using authentication middleware
- Store and verify tokens securely
- Build secure backend authentication flows
- Identify common authentication security risks
- Use AI-generated security code critically and responsibly

- Implement token expiration and role-based access control concepts

Technologies Used

- technology: Node.js, purpose: JavaScript runtime
- technology: Express.js, purpose: Backend framework
- technology: PostgreSQL, purpose: Relational database
- technology: Bcrypt, purpose: Password hashing
- technology: Jsonwebtoken, purpose: JWT token generation
- technology: Postman, purpose: API testing
- technology: dotenv, purpose: Environment variable management

Authentication vs Authorization

These two concepts are commonly confusing but serve different purposes.

- Authentication: Verifying who a user is
- Authorization: Determining what a user can access

Examples:

- Login with email/password = Authentication
- Access admin dashboard = Authorization

Authentication happens first. Authorization happens after identity is verified.

Why Passwords Must Never Be Stored Plain Text

Storing plain-text passwords is a critical security vulnerability.
Example of BAD practice:

```
password = ‘mypassword123’
```

If the database is compromised:

- all user accounts become exposed
- attackers may reuse passwords elsewhere
- legal and security consequences may occur

Professional applications always hash passwords before storage.

What is Password Hashing?

Hashing converts a password into an irreversible encrypted value.

Example:

```
password123
```

becomes:

```
$2b$10$8Jj3s7...
```

Properties of hashing:

- one-way transformation
- cannot easily recover original password
- protects user credentials
- industry-standard security practice

What is JWT?

JWT, JSON Web Token, is commonly used for stateless authentication.

A JWT token:

- stores encoded user information
- verifies authenticated users
- is sent with future API requests
- allows protected route access

JWT Authentication Flow

Step-by-step process:

1. User registers account
2. Password is hashed before storage
3. User logs in
4. Server validates credentials
5. Server generates JWT token
6. Client stores token
7. Client sends token in future requests
8. Server verifies token
9. Protected resources become accessible

JWT Structure

A JWT contains three sections: HEADER.PAYLOAD.SIGNATURE

Example:

```
xxxxx.yyyyy.zzzzz
```

- Section: header, purpose: Algorithm information
- Section: Payload, purpose: User data
- Section: Signature, purpose: Security verification

Install Required Packages

Open terminal.

Install:

```
npm install bcrypt jsonwebtoken
--
```

Package Purposes

- Package: bcrypt, purpose: Password hashing
- Package: jsonwebtoken, purpose: JWT token generation and verification

Project Structure

Update project structure:

```
project/
|
├── middleware/
|   └── authMiddleware.js
|
├── routes/
|   └── authRoutes.js
|
├── controllers/
|   └── authController.js
|
└── .env
```

Lab Procedure

Step 1 — Add JWT Secret

Inside:
`.env`

Add:
```
JWT_SECRET=my_super_secret_key
```

Why JWT Secrets Matter
JWT secrets are used to:

1. sign tokens
2. verify token authenticity
3. prevent token tampering

Never expose secrets publicly. Never hardcode secrets inside source code.

Step 2 — Create Register Endpoint

Inside:
`authRoutes.js`

Add:
```
const express = require('express');
const router = express.Router();
const bcrypt = require('bcrypt');
const db = require('../db/db');

router.post('/auth/register', async (req, res) => {
try {
     const { email, password } = req.body;

     const hashed = await bcrypt.hash(password, 10);

     const result = await db.query(
     `INSERT INTO users (email, password_hash)
     VALUES ($1, $2)
     RETURNING id, email`,
     [email, hashed]
     );

     res.status(201).json(result.rows[0]);
} catch (error) {
```

```
    console.error(error);
    res.status(500).json({ error: 'Server error' });
    }
});
```

Understanding bcrypt.hash()

```
bcrypt.hash(password, 10)
```

The second parameter:
10

represents salt rounds.

More rounds:

- stronger security
- slower processing

Typical production values:
10
12
14

Why Salting Matters

Salting adds random data before hashing.
Benefits:

- prevents rainbow table attacks
- makes identical passwords generate different hashes
- improves security significantly

Screenshot --Successful user registration request in Postman

Step 3 — Create Login Endpoint

Add:

```
const jwt = require('jsonwebtoken');
require('dotenv').config();
```

Then:

```
router.post('/auth/login', async (req, res) => {
```

```
try {
     const { email, password } = req.body;
          const result = await db.query(
          'SELECT * FROM users WHERE email = $1',
          [email]
          );

     const user = result.rows[0];

     if (!user) {
          return res.status(401).json({
          error: 'Invalid credentials'
          });
     }

     const valid = await bcrypt.compare(
          password,
          user.password_hash
     );

     if (!valid) {
          return res.status(401).json({
          error: 'Invalid credentials'
          });
     }

     const token = jwt.sign(
          { userId: user.id },
          process.env.JWT_SECRET
     );

     res.json({ token });

} catch (error) {
     console.error(error);
     res.status(500).json({ error: 'Server error' });
     }
});
```

Understanding bcrypt.compare()

```
bcrypt.compare(password, user.password_hash)
```

This safely compares:

- user-entered password
- stored hashed password

Never compare hashes manually. Always use bcrypt methods.

Understanding jwt.sign()

```
jwt.sign({ userId: user.id }, SECRET)
This generates a token containing:
{
     userId: user.id
}
```

The token is cryptographically signed using the secret key.

Example JWT Response

```
{
     "token": "eyJhbGciOiJIUzI1Ni..."
}
```

This token will be used for future protected requests.
Screenshot --Login request returning JWT token

Step 4 — Create Authentication Middleware

Create:

```
middleware/authMiddleware.js
```

Add:

```
const jwt = require('jsonwebtoken');
require('dotenv').config();

function auth(req, res, next) {
try {
     const authHeader = req.headers.authorization;

     if (!authHeader) {
          return res.status(401).json({
               error: 'Access denied'
```

```
            });
        }

        const token = authHeader.split(' ')[1];

        const verified = jwt.verify(
            token,
            process.env.JWT_SECRET
        );

        req.user = verified;

        next();

    } catch (error) {
        res.status(401).json({
            error: 'Invalid token'
        });
    }
}

module.exports = auth;
```

Understanding Middleware

Middleware functions:

- run before route handlers
- process requests centrally
- can stop requests
- can modify request objects

Middleware is heavily used in professional backend applications.

Understanding Authorization Headers

Protected requests usually send:
Authorization: Bearer TOKEN
Example:
Authorization: Bearer eyJhbGciOiJIUzI1Ni...

Step 5 — Protect Routes
Inside:
`taskRoutes.js`

Add:
```
const auth = require('../middleware/authMiddleware');
```

Protect route:
```
router.get('/tasks', auth, async (req, res) => {
```

What Happens Now?
When a request accesses:
GET /tasks
The server will:

1. Run auth middleware
2. Verify token
3. Allow or deny access
4. Continue to route handler if valid

Screenshot --Protected route returning data with valid token
Screenshot --Unauthorized request failing without token

Testing Protected APIs in Postman

Inside Postman:

1. Open Headers tab
2. Add:

Authorization
Value:
```
Bearer YOUR_TOKEN
```

Common Authentication Errors

- Error: Invalid token, cause: Wrong secret or malformed token
- Error: Access denied, cause: Missing token
- Error: JWT expired, cause: Expired token
- Error: Invalid credentials, cause: Wrong email/password
- Error: Cannot hash password, cause: bcrypt missing

Security Best Practices

Professional backend systems should:

1. hash all passwords
2. never expose secrets publicly
3. use HTTPS in production
4. validate all user input
5. use parameterized SQL queries
6. implement token expiration
7. avoid storing sensitive data inside JWT payloads

Why JWT is Popular

JWT authentication is widely used because:

- stateless architecture
- scalable systems
- frontend-friendly
- mobile-friendly
- microservice-compatible
- cloud deployment friendly

JWT Limitations

JWT also has tradeoffs:

- difficult token revocation
- larger payload size
- token leakage risks
- expiration management complexity

Developers must understand both advantages and limitations.

AI-Assisted Development Activity

Modern developers increasingly use AI tools to assist with:

- authentication logic
- middleware generation
- debugging
- security suggestions
- API structure

Examples:

- ChatGPT
- GitHub Copilot
- Claude

However, security-related AI-generated code must always be reviewed carefully.

AI Task Prompt

Use an AI tool and prompt:

"Add JWT authentication middleware to protect Express routes."

Required AI Analysis

After generating AI output:

- identify security issues
- verify middleware logic
- review token verification
- validate error handling
- improve insecure code
- compare AI output vs your implementation

Important Security Principle

Authentication and security systems should never be blindly copied from AI.
Developers must:

- understand security logic
- validate generated code
- identify vulnerabilities
- apply industry best practices
- test security thoroughly

Bonus challenges

Bonus Challenge 1 — Token Expiration

Improve token generation:

```
const token = jwt.sign(
{ userId: user.id },
process.env.JWT_SECRET,
{ expiresIn: '1h' }
);
```

Why Token Expiration Matters

Benefits:

- limits stolen token usage
- improves security
- reduces long-term access risks
- industry-standard practice

Bonus Challenge 2 — Role-Based Access Control

Add role column:

```
ALTER TABLE users
ADD COLUMN role VARCHAR(20);
```

Possible values:

- admin
- user

Example Authorization Middleware

```
function adminOnly(req, res, next) {
if (req.user.role !== 'admin') {
return res.status(403).json({
error: 'Forbidden'
});
}
next();
}
```

Authentication vs Authorization Review

- authentication: Who are you?, authorization: What can you access?
- Authentication: Login process, authorization: Permission control
- Authentication: JWT verification, authorization: Role checking
- Authentication: Identity validation, authorization: Resource restrictions

Final Deliverables

Submit:

- Updated Node.js project
- Register endpoint
- Login endpoint
- JWT authentication middleware
- Protected API routes
- Screenshots of Postman testing
- AI-generated authentication example
- AI security analysis and corrections
- Bonus challenge implementations (optional)

Submission Checklist

- bcrypt installed
- jsonwebtoken installed
- Password hashing implemented
- Login endpoint working
- JWT token generated
- Auth middleware created
- Protected routes tested
- AI task completed
- Token expiration added
- Role-based access added

Conclusion

In this lab, you implemented a secure authentication system using bcrypt and JWT within a Node.js and Express backend application. You learned how passwords are securely stored, how authentication middleware protects routes, and how token-based authentication works in modern backend systems.

You also explored AI-assisted security development workflows while critically reviewing and improving AI-generated authentication code.

This lab establishes the security foundation for upcoming topics including testing, API documentation, Docker, and deployment.